EEL 4742C: Embedded Systems
Name: Ryan Ramdihal
Lab 5: LCD Display

Introduction:

In this lab, our first objective is to populate an array within the provided sample code to store shapes representing digits 0 through 9. Utilizing a straightforward arrangement where each index of the array corresponds to its specific digit streamlines the process of retrieving and displaying these shapes later in the program. To accomplish this, we first determine the binary values representing the shapes of each digit, which are then converted into hexadecimal format for enhanced readability and accessibility. Following this, we modify the code to print a 16-bit unsigned integer to the LCD display, ensuring that there are no non-leading zeros displayed. This is achieved through a function that iterates through each digit of the input number, assigning it to the appropriate LCD segment based on its position within the number.

5.1 Printing on the LCD

The primary objective of this lab is to populate an array within the provided sample code. This array is designed to store the shapes representing digits 0 through 9. Each index of the array corresponds to its specific digit: This arrangement simplifies the process of retrieving the shapes and displaying them later in the program. To achieve this, we need to determine the binary values representing the shapes of each digit. Once we have these binary values, we convert them into hexadecimal format and fill in the array accordingly. Using hexadecimal notation enhances the readability and accessibility of the code.

The next objective is to now modify the code to print a 16 bit unsigned integer to the LCD display. There should not be any non leading zeros in the display. To combat this, the function operates by iterating through each digit of the input number and assigning it to the appropriate LCD segment based on the position within the number. The function stores that extracted digit separately by using the mod 10 operation and sets the lcd display to that number's array with the corresponding index. All inside the do-while loop that places that digit in the display correct LCD position.

5.1 Print 430 Code:

```
// Sample code that prints 430 on the LCD monitor
#include <msp430fr6989.h>
#define redLED BIT0 // Red at P1.0
#define greenLED BIT7 // Green at P9.7
void Initialize_LCD();
// The array has the shapes of the digits (0 to 9)
// Complete this array...
const unsigned char LCD_Shapes[10] = {0xFC, 0x60, 0xDB, 0xF3, 0x67, 0xB7, 0xBF, 0xE0, 0xFF, 0xE7};
int main(void) {
volatile unsigned int n;
WDTCTL = WDTPW | WDTHOLD; // Stop WDT
PM5CTL0 &= ~LOCKLPM5; // Enable GPIO pins
P1DIR |= redLED; // Pins as output
P9DIR |= greenLED;
P1OUT |= redLED; // Red on
```

```
P9OUT &= ~greenLED; // Green off
// Initializes the LCD_C module
Initialize_LCD();
// The line below can be used to clear all the segments
//LCDCMEMCTL = LCDCLRM; // Clears all the segments
// Display 430 on the rightmost three digits
LCDM19 = LCD_Shapes[4];
LCDM15 = LCD_Shapes[3];
LCDM8 = LCD_Shapes[0];
// Flash the red LED
for(;;) {
for(n=0; n<=60000; n++) {} // Delay loop
P1OUT ^= redLED;
}
return 0;
}
//*************************************************************
// Initializes the LCD_C module
// *** Source: Function obtained from MSP430FR6989's Sample Code ***
void Initialize_LCD() {
PJSEL0 = BIT4 | BIT5; // For LFXT
// Initialize LCD segments 0 - 21; 26 - 43
LCDCPCTL0 = 0xFFFF;
LCDCPCTL1 = 0xFC3F;
LCDCPCTL2 = 0x0FFF;
// Configure LFXT 32kHz crystal
CSCTL0_H = CSKEY >> 8; // Unlock CS registers
CSCTL4 &= ~LFXTOFF; // Enable LFXT
do {
CSCTL5 &= ~LFXTOFFG; // Clear LFXT fault flag
SFRIFG1 &= ~OFIFG;
}while (SFRIFG1 & OFIFG); // Test oscillator fault flag
CSCTL0_H = 0; // Lock CS registers
// Initialize LCD_C
// ACLK, Divider = 1, Pre-divider = 16; 4-pin MUX
LCDCCTL0 = LCDDIV__1 | LCDPRE__16 | LCD4MUX | LCDLP;
// VLCD generated internally,
// V2-V4 generated internally, v5 to ground
// Set VLCD voltage to 2.60v
// Enable charge pump and select internal reference for it
LCDCVCTL = VLCD_1 | VLCDREF_0 | LCDCPEN;
LCDCCPCTL = LCDCPCLKSYNC; // Clock synchronization enabled
LCDCMEMCTL = LCDCLRM; // Clear LCD memory
//Turn LCD on
LCDCCTL0 |= LCDON;
return;
}
```

5.1 Print number 0-65535 code:

```
/// Sample code modified that can print 0-65535
#include <msp430fr6989.h>
#define redLED BIT0 // Red at P1.0
```

```
#define greenLED BIT7 // Green at P9.7
void Initialize_LCD();
void lcd_write_uint16();
//8-Segment display numbers
const unsigned char LCD_Shapes[10] = {0xFC, 0x60, 0xDB, 0xF3, 0x67, 0xB7, 0xBF, 0xE0, 0xFF, 0xE7};
int main(void) {
   volatile unsigned int n;
   WDTCTL = WDTPW | WDTHOLD; // Stop WDT
   PM5CTL0 &=  ~LOCKLPM5; // Enable GPIO pins
   P1DIR |= redLED; // Pins as output
   P9DIR |= greenLED;
   P1OUT |= redLED; // Red on
   P9OUT &=  ~greenLED; // Green off
   // Initializes the LCD_C module
   Initialize_LCD();
   // The line below can be used to clear all the segments
   //LCDCMEMCTL = LCDCLRM; // Clears all the segments
   // Display given number
   //0-65535, ((2^16)-1)
   unsigned int num = 530;
   lcd_write_uint16(num);
   // Flash the red LED
   for(;;) {
      for(n=0; n<=60000; n++) {} // Delay loop
      P1OUT ^= redLED;
   }
}
void lcd_write_uint16(unsigned int num){
   int i = 6, digit;
      do {
         digit = num % 10; // remainder is that digit
         if (i == 6)
            LCDM8 = LCD_Shapes[digit]; //A6
         else if (i == 5)
            LCDM15 = LCD_Shapes[digit]; //A5
         else if (i == 4)
            LCDM19 = LCD_Shapes[digit]; //A4
         else if (i == 3)
            LCDM4 = LCD_Shapes[digit]; //A3
         else if (i == 2)
            LCDM6 = LCD_Shapes[digit]; //A2
         else if (i == 1)
            LCDM10 = LCD_Shapes[digit]; //A1
         num = num / 10;// remove last digit and continue
         i--;
      }while (num > 0);
   return;
}
void Initialize_LCD() {
   PJSEL0 = BIT4 | BIT5; // For LFXT
   // Initialize LCD segments 0 - 21; 26 - 43
   LCDCPCTL0 = 0xFFFF;
   LCDCPCTL1 = 0xFC3F;
   LCDCPCTL2 = 0x0FFF;
```

```
    // Configure LFXT 32kHz crystal
    CSCTL0_H = CSKEY >> 8; // Unlock CS registers
    CSCTL4 &= ~LFXTOFF; // Enable LFXT
    do {
        CSCTL5 &= ~LFXTOFFG; // Clear LFXT fault flag
        SFRIFG1 &= ~OFIFG;
    }while (SFRIFG1 & OFIFG); // Test oscillator fault flag
    CSCTL0_H = 0; // Lock CS registers
    // Initialize LCD_C
    // ACLK, Divider = 1, Pre-divider = 16; 4-pin MUX
    LCDCCTL0 = LCDDIV__1 | LCDPRE__16 | LCD4MUX | LCDLP;
    // VLCD generated internally,
    // V2-V4 generated internally, v5 to ground
    // Set VLCD voltage to 2.60v
    // Enable charge pump and select internal reference for it
    LCDCVCTL = VLCD_1 | VLCDREF_0 | LCDCPEN;
    LCDCCPCTL = LCDCPCLKSYNC; // Clock synchronization enabled
    LCDCMEMCTL = LCDCLRM; // Clear LCD memory
    //Turn LCD on
    LCDCCTL0 |= LCDON;
    return;
}
```

5.2 Implementing a Counter

Our next task is to develop a 16-bit counter application that counts from 0 to 65535. This will be achieved using interrupts based on a 32KHz crystal oscillator. The system will feature two buttons: S1 to reset the counter and S2 to increment it by 1000. Upon initialization, the LCD and the timer will be set up for up mode operation. Both buttons will be configured to trigger interrupts, and Timer_A0 will be configured to generate an interrupt every second by setting TACCRO to 32767.

In the interrupt service routine for the buttons, pressing Button 1 will reset the counter n to 0, while pressing Button 2 will add 1000 to the current count. In the Timer_A0 interrupt service routine, n will be incremented by 1, and the updated value will be displayed on the LCD.

5.2 Code:

```
#include <msp430fr6989.h>
#define redLED BIT0 // Red at P1.0
#define greenLED BIT7 // Green at P9.7
#define BUT1 BIT1 // P1.1
#define BUT2 BIT2 //P1.2
void config_ACLK_to_32KHz_crystal();
void Initialize_LCD();
void lcd_write_uint16();
//8-Segment display numbers
const unsigned char LCD_Shapes[10] = {0xFC, 0x60, 0xDB, 0xF3, 0x67, 0xB7, 0xBF, 0xE0, 0xFF, 0xE7};
unsigned int n = 0;
int main(void) {
volatile unsigned int n;
WDTCTL = WDTPW | WDTHOLD; // Stop WDT
PM5CTL0 &= ~LOCKLPM5; // Enable GPIO pins
//LED setup
```

```
P1DIR |= redLED; // Pins as output
P9DIR |= greenLED;
P1OUT |= redLED; // Red on
P9OUT &=  ~greenLED; // Green off
// Configure the buttons for interrupts
P1DIR &=  ~(BUT1|BUT2); // 0: input
P1REN |= (BUT1|BUT2); // 1: enable built-in resistors
P1OUT |= (BUT1|BUT2); // 1: built-in resistor is pulled up to Vcc
P1IES |= (BUT1|BUT2); // 1: interrupt on falling edge (0 for rising edge)
P1IFG &=  ~(BUT1|BUT2); // 0: clear the interrupt flags
P1IE |= (BUT1|BUT2); // 1: enable the interrupts
// Configure Channel 0 for up mode with interrupts
TA0CCR0 = 32767; // 1 second @ 32 KHz
TA0CCTL0 |= CCIE; // Enable Channel 0 CCIE bit
TA0CCTL0 &= ~CCIFG; // Clear Channel 0 CCIFG bit
// Timer_A: ACLK, div by 1, up mode, clear TAR
TA0CTL = TASSEL_1 | ID_0 | MC_1 | TACLR;
// Enable the global interrupt bit (call an intrinsic function)
_enable_interrupts();
// Initializes the LCD_C module
Initialize_LCD();
// The line below can be used to clear all the segments
//LCDCMEMCTL = LCDCLRM; // Clears all the segments
// Display 0;
lcd_write_uint16(n);
// Flash the red LED
for(;;) {}
}
void lcd_write_uint16(unsigned int n){
   int digit;
   digit = n % 10;//extract least significant digit
   LCDM8 = LCD_Shapes[digit]; //A 6
   n = n / 10; //remove least significant digit
   digit = n % 10;
   LCDM15 = LCD_Shapes[digit]; //A 5
   n = n / 10;
   digit = n % 10;
   LCDM19 = LCD_Shapes[digit]; //A 4
   n = n / 10;
   digit = n % 10;
   LCDM4 = LCD_Shapes[digit]; //A 3
   n = n / 10;
   digit = n % 10;
   LCDM6 = LCD_Shapes[digit]; //A 2
   n = n / 10;
   digit = n % 10;
   //LCDM10 = LCD_Shapes[digit]; //A1, not used
   return;
}
void Initialize_LCD() {
PJSEL0 = BIT4 | BIT5; // For LFXT
// Initialize LCD segments 0 - 21; 26 - 43
LCDCPCTL0 = 0xFFFF;
LCDCPCTL1 = 0xFC3F;
```

```
LCDCPCTL2 = 0x0FFF;
// Configure LFXT 32kHz crystal
CSCTL0_H = CSKEY >> 8; // Unlock CS registers
CSCTL4 &=  ~LFXTOFF; // Enable LFXT
do {
CSCTL5 &=  ~LFXTOFFG; // Clear LFXT fault flag
SFRIFG1 &=  ~OFIFG;
}while (SFRIFG1 & OFIFG); // Test oscillator fault flag
CSCTL0_H = 0; // Lock CS registers
// Initialize LCD_C
// ACLK, Divider = 1, Pre-divider = 16; 4-pin MUX
LCDCCTL0 = LCDDIV__1 | LCDPRE__16 | LCD4MUX | LCDLP;
// VLCD generated internally,
// V2-V4 generated internally, v5 to ground
// Set VLCD voltage to 2.60v
// Enable charge pump and select internal reference for it
LCDCVCTL = VLCD_1 | VLCDREF_0 | LCDCPEN;
LCDCCPCTL = LCDCPCLKSYNC; // Clock synchronization enabled
LCDCMEMCTL = LCDCLRM; // Clear LCD memory
//Turn LCD on
LCDCCTL0 |= LCDON;
return;
}
void config_ACLK_to_32KHz_crystal() {
// By default, ACLK runs on LFMODCLK at 5MHz/128 = 39 KHz
// Reroute pins to LFXIN/LFXOUT functionality
PJSEL1 &= ~BIT4;
PJSEL0 |= BIT4;
// Wait until the oscillator fault flags remain cleared
CSCTL0 = CSKEY; // Unlock CS registers
do {
CSCTL5 &= ~LFXTOFFG; // Local fault flag
SFRIFG1 &= ~OFIFG; // Global fault flag
} while((CSCTL5 & LFXTOFFG) != 0);
CSCTL0_H = 0; // Lock CS registers
return;
}
#pragma vector = PORT1_VECTOR // Define an interrupt service routine for Port 1 interrupts
__interrupt void Port1_ISR() { // Start of the Port 1 interrupt service routine
   //Detect button1
   if((P1IFG & BUT1) == BUT1){ // Check if button 1 triggered the interrupt
      n = 0; // Reset the displayed number to 0
      lcd_write_uint16(n); // Write the updated number to the LCD
      __delay_cycles (100000); // Delay for debounce
      P1IFG &=  ~(BUT1); // Clear the interrupt flag for button 1
   }
   //Detect button2
   if((P1IFG & BUT2) == BUT2){ // Check if button 2 triggered the interrupt
      n += 1000; // Increment the displayed number by 1000
      lcd_write_uint16(n); // Write the updated number to the LCD
      __delay_cycles (100000); // Delay for debounce
      P1IFG &=  ~(BUT2); // Clear the interrupt flag for button 2
   }
} // End of the Port 1 interrupt service routine
```

```
#pragma vector = TIMER0_A0_VECTOR // Define an interrupt service routine for Timer A0, CCIFG0 vector
__interrupt void T0A0_ISR() { // Start of the Timer A0 interrupt service routine
   //Increment n
   n++; // Increment the displayed number
   //Print n
   lcd_write_uint16(n); // Write the updated number to the LCD
   P1OUT ^= redLED; // Toggle the red LED
   P9OUT ^= greenLED;
} // End of the Timer A0 interrupt service routine
```

Conclusion:

In conclusion, this lab provided skills in array utilization, with practical applications in displaying numerical values on an LCD screen. Through the utilization of hexadecimal notation and the arrangement of LCD segments, achieving a coherent and easily understandable display. Furthermore, the development of the 16-bit counter application showcased the integration of interrupts, buttons, and timers, resulting in a responsive interface for monitoring numeric values.

Student Q & A

1. Explain whether this statement is true or false. If false, explain the correct operation. “An LCD segment works just like a colored LED. It’s turned on/off by writing either digital high/low to it, respectively”.
False. The LCD controller manages the voltages for specific segments on the display.
2. What is the name of the LCD controller that interfaces the LCD display of our board? Is the LCD controller located on the display module or in the microcontroller?
LCD C module. The LCD controller is located in the microcontroller.

3. In what multiplexing configuration is the LCD module wired (2-way, 4-way, etc)? What does this mean regarding the number of pins used at the microcontroller?
4-way. Multiple LCD segments can use the same pin.